

Agentic AI Pipeline

A simple agentic AI system that routes user queries to data, math, or general agents and uses tools when required.

Note: This pipeline architecture is designed to work with Gemini. For local development and testing, Ollama with `llama3.1:latest` is used to avoid API quota limitations. The agent architecture and workflow remain the same.

Import required functions

```
In [1]: from agents import run_agent
        from tools import calculator, analyze_data
```

Test data query

```
In [2]: query = "Find the average sales from sales.csv"

        route, result, trajectory = await run_agent(query)

        print("Route:", route)
        print("Answer:", result.messages[-1].content)
        print("Trajectory:", trajectory)
```

Route: DATA

Answer: 1800.0

Trajectory: {'query': 'Find the average sales from sales.csv', 'route': 'DATA', 'attempt': 1, 'status': 'success'}

Test math query

```
In [3]: query = "Calculate 125 multiplied by 8"

        route, result, trajectory = await run_agent(query)

        print("Route:", route)
        print("Answer:", result.messages[-1].content)
        print("Trajectory:", trajectory)
```

Route: MATH

Answer: 1000.0

Trajectory: {'query': 'Calculate 125 multiplied by 8', 'route': 'MATH', 'attempt': 1, 'status': 'success'}

Test general query

```
In [4]: query = "What is overfitting in machine learning?"

        route, result, trajectory = await run_agent(query)
```

```
print("Route:", route)
print("Answer:", result.messages[-1].content)
print("Trajectory:", trajectory)
```

Route: GENERAL.

Answer: In simple terms, overfitting occurs when a model is too good at memorizing the training data, but not able to generalize well to new, unseen data.

Think of it like this: Imagine you're trying to recognize your friends' faces. If you only look at pictures of them from one angle, with the same lighting and background, you might become very good at recognizing their face in those specific conditions. But if you go out into a different environment or see them from a different angle, you won't be able to recognize them as easily.

Similarly, an overfitting model is too focused on the training data it's seen, rather than learning the underlying patterns and relationships that apply to new situations. This can lead to poor performance when the model is applied to real-world data.

Does this make sense?

Trajectory: {'query': 'What is overfitting in machine learning?', 'route': 'GENERAL.', 'attempt': 1, 'status': 'success'}

Test multiple queries

```
In [5]: queries = [
        "Find the average sales from sales.csv",
        "Calculate 125 multiplied by 8",
        "What is overfitting in machine learning?"
    ]

    results = []

    for query in queries:
        route, result, trajectory = await run_agent(query)

        results.append({
            "query": query,
            "route": route,
            "answer": result.messages[-1].content,
            "completed": trajectory["status"] == "success"
        })

    for item in results:
        print("Query:", item["query"])
        print("Route:", item["route"])
        print("Completed:", item["completed"])
        print()
```

Query: Find the average sales from sales.csv

Route: DATA

Completed: True

Query: Calculate 125 multiplied by 8

Route: MATH

Completed: True

Query: What is overfitting in machine learning?

Route: GENERAL

Completed: True

Check completion rate

```
In [6]: completion_rate = (  
    sum(item["completed"] for item in results) / len(results)  
    ) * 100  
  
print("Completion Rate:", completion_rate, "%")
```

Completion Rate: 100.0 %

Measure response time

```
In [7]: import time  
  
timing_results = []  
  
for query in queries:  
    start = time.time()  
  
    route, result, trajectory = await run_agent(query)  
  
    elapsed = time.time() - start  
  
    timing_results.append({  
        "query": query,  
        "route": route,  
        "completed": trajectory["status"] == "success",  
        "time_seconds": round(elapsed, 2),  
        "attempts": trajectory["attempt"]  
    })  
  
timing_results
```

```
Out[7]: [{'query': 'Find the average sales from sales.csv',
        'route': 'DATA',
        'completed': True,
        'time_seconds': 2.79,
        'attempts': 1},
        {'query': 'Calculate 125 multiplied by 8',
        'route': 'MATH',
        'completed': True,
        'time_seconds': 2.6,
        'attempts': 1},
        {'query': 'What is overfitting in machine learning?',
        'route': 'GENERAL.',
        'completed': True,
        'time_seconds': 9.69,
        'attempts': 1}]
```

Performance summary

```
In [8]: average_time = sum(
        item["time_seconds"] for item in timing_results
    ) / len(timing_results)

print("Completion Rate:", completion_rate, "%")
print("Average Response Time:", round(average_time, 2), "seconds")
```

Completion Rate: 100.0 %
Average Response Time: 5.03 seconds

Check sales statistics

```
In [9]: average = analyze_data("sales.csv", "sales", "average")
        maximum = analyze_data("sales.csv", "sales", "max")
        minimum = analyze_data("sales.csv", "sales", "min")

print("Average:", average)
print("Maximum:", maximum)
print("Minimum:", minimum)
```

Average: 1800.0
Maximum: 2400.0
Minimum: 1200.0

View test results

```
In [10]: import pandas as pd

         results_df = pd.DataFrame(timing_results)

         results_df
```

Out[10]:

	query	route	completed	time_seconds	attempts
0	Find the average sales from sales.csv	DATA	True	2.79	1
1	Calculate 125 multiplied by 8	MATH	True	2.60	1
2	What is overfitting in machine learning?	GENERAL	True	9.69	1

Test parallel queries

In [11]:

```
import asyncio
import time

async def run_parallel():
    start = time.time()

    outputs = await asyncio.gather(
        *[run_agent(query) for query in queries]
    )

    elapsed = time.time() - start

    return outputs, elapsed

parallel_results, parallel_time = await run_parallel()

print("Parallel Time:", round(parallel_time, 2), "seconds")

for query, (route, result, trajectory) in zip(queries, parallel_results):
    print("Query:", query)
    print("Route:", route)
    print("Completed:", trajectory["status"] == "success")
    print()
```

Parallel Time: 12.27 seconds

Query: Find the average sales from sales.csv

Route: DATA

Completed: True

Query: Calculate 125 multiplied by 8

Route: MATH

Completed: True

Query: What is overfitting in machine learning?

Route: THE CATEGORY FOR THIS QUESTION IS:

GENERAL

Completed: True

Compare execution time

In [12]:

```
sequential_time = sum(
    item["time_seconds"] for item in timing_results
)
```

```
print("Sequential Time:", round(sequential_time, 2), "seconds")
print("Parallel Time:", round(parallel_time, 2), "seconds")

if parallel_time < sequential_time:
    print("Parallel execution was faster.")
else:
    print("Sequential execution was faster.")
```

Sequential Time: 15.08 seconds

Parallel Time: 12.27 seconds

Parallel execution was faster.

Agent workflow

```
In [13]: workflow = """
User Query
    ↓
Planner
    ↓
Conditional Router
    ├── DATA → Data Agent → CSV Tool
    ├── MATH → Math Agent → Calculator
    └── GENERAL → General Agent
    ↓
Retry Handling
    ↓
Final Response
"""

print(workflow)
```

```
User Query
    ↓
Planner
    ↓
Conditional Router
    ├── DATA → Data Agent → CSV Tool
    ├── MATH → Math Agent → Calculator
    └── GENERAL → General Agent
    ↓
Retry Handling
    ↓
Final Response
```

Final evaluation

```
In [14]: print("Agentic AI Pipeline Evaluation")
print("-" * 35)
print(f"Completion Rate: {completion_rate:.1f}%")
print(f"Average Response Time: {average_time:.2f} seconds")
print(f"Queries Tested: {len(queries)}")
print(f"Parallel Execution Time: {parallel_time:.2f} seconds")
```

Agentic AI Pipeline Evaluation

Completion Rate: 100.0%

Average Response Time: 5.03 seconds

Queries Tested: 3

Parallel Execution Time: 12.27 seconds

Observations

The pipeline successfully routed the test queries to the correct agents. Data and math queries used their respective tools, while general questions were handled directly by the general agent.

Future improvements

The system can be improved by adding more tools, supporting multiple datasets, improving query routing, and adding better error handling and evaluation.